

End-to-End Data Pipeline Development

Enhanced Edition with Container Orchestration and Comprehensive Log Management

This assignment challenges you to construct a comprehensive data pipeline that mirrors real-world enterprise architectures. You'll gain hands-on experience with the complete data lifecycle—from generation through to reporting—whilst mastering industry-standard tools including Apache NiFi, Kafka, Hadoop, and container orchestration platforms. The objective is to develop both technical proficiency and architectural thinking necessary for production-scale data engineering.

Core Pipeline Components and Technical Requirements

01 - Container Orchestration Infrastructure [NEW REQUIREMENT]

Deploy the entire ecosystem using containerization:

- **Container Platform Choice:** Implement your data pipeline infrastructure using either Docker Compose or Kubernetes
 - For Docker Compose: Minimum 5 interconnected services
 - For Kubernetes: Minimum 3-node cluster (can use Minikube, K3s, or cloud-managed Kubernetes)
- **Containerized Services Must Include:**
 - Apache NiFi (data ingestion layer)
 - Apache Kafka cluster (broker, zookeeper)
 - Apache Hadoop ecosystem (HDFS, YARN, MapReduce)
 - Custom microservices (minimum 3 services) that generate application logs
 - Database service (e.g., PostgreSQL, MySQL, or MongoDB for metadata)
 - Monitoring stack (optional: Prometheus + Grafana)
- **Service Configuration Requirements:**
 - All services must be configured via environment variables or ConfigMaps/Secrets
 - Implement proper networking between containers (use bridge networks for Docker, Services for Kubernetes)
 - Configure persistent volumes for stateful services (Kafka, Hadoop, databases)
 - Implement resource limits (CPU, memory) for each container
 - Use health checks and readiness probes for all services

02 - Multi-Source Log Generation System [ENHANCED REQUIREMENT]

Design and implement a comprehensive logging ecosystem:

A. Service Application Logs

- Deploy minimum 3 microservices (can be simple web applications, APIs, or data processors)
- Each service must generate structured application logs including:
 - HTTP access logs (requests, responses, status codes)
 - Application error logs (exceptions, stack traces)
 - Business logic logs (transactions, user activities)
 - Performance metrics (response times, query durations)
- Implement different log formats: JSON, Common Log Format (CLF), custom structured formats
- Generate realistic traffic patterns (sustained 100+ requests/minute across all services)

B. System Logs (Syslog)

- Configure container/host-level syslog collection
- Capture kernel messages, authentication logs, and system events
- Route syslog messages to centralized collection points
- Implement severity-based filtering (INFO, WARNING, ERROR, CRITICAL)

C. Event Logs

- Generate discrete event streams representing:
 - User authentication events (login, logout, failed attempts)
 - Service lifecycle events (startup, shutdown, configuration changes)
 - Security events (access violations, permission changes)
 - Business events (orders, payments, registrations)
- Include event metadata: timestamp, source, severity, correlation IDs

D. Detailed Service Records

- Database query logs with execution times
- Cache hit/miss ratios and performance metrics
- Inter-service communication traces (service mesh style)
- Resource utilization metrics (CPU, memory, disk I/O per container)
- Custom business metrics relevant to your services

Log Volume Targets:

- Sustained generation: 1,000-5,000 log entries per minute
- Peak capacity: 10,000+ entries per minute
- Multiple concurrent log streams (minimum 5 distinct sources)

03 - Data Ingestion and Transfer with Apache NiFi [ENHANCED]

Configure Apache NiFi to collect logs from containerized services:

- **Multi-Source Log Collection:**

- Implement processors to collect logs from:
 - Container stdout/stderr streams
 - Log files mounted via volumes
 - Syslog receivers (UDP/TCP listeners)
 - HTTP endpoints (webhook-style log delivery)
 - Direct database connections for query logs
 - **Data Flow Requirements:**
 - Create separate NiFi process groups for each log type
 - Implement content-based routing based on log severity and source
 - Add data validation and schema enforcement processors
 - Configure back-pressure thresholds appropriate for log volume
 - Implement retry logic with exponential backoff for failed transfers
 - **Log Enrichment:**
 - Add metadata tags (environment, service name, pod/container ID)
 - Append correlation IDs for distributed tracing
 - Normalize timestamps to UTC
 - Perform initial parsing and field extraction
 - **Monitoring and Reliability:**
 - Configure NiFi bulletins and provenance tracking
 - Implement flow file expiration policies
 - Set up connection queuing with appropriate prioritization
 - Monitor data lineage from source to destination
-

04 - Stream Processing with Apache Kafka [ENHANCED]

Integrate Apache Kafka as the distributed streaming backbone:

- **Topic Architecture:**
 - Create dedicated topics for each log type:
 - `service-logs-raw` (application logs)
 - `system-logs-syslog` (syslog entries)
 - `event-logs-structured` (event streams)
 - `service-metrics` (performance and resource metrics)
 - Implement separate topics for transformed/enriched data
 - Configure appropriate partition counts (minimum 3 partitions per topic for parallelism)
- **Kafka Configuration:**
 - Set replication factor of 2 or higher for fault tolerance
 - Configure retention policies:
 - Time-based: 7 days for raw logs
 - Size-based: 10GB per partition maximum
 - Implement log compaction for keyed event streams
 - Tune batch sizes and compression (e.g., snappy or lz4)
- **Producer and Consumer Integration:**
 - NiFi as Kafka producer (using PublishKafka processors)
 - Implement consumer groups for parallel processing
 - Configure acknowledgment modes (at-least-once delivery)
 - Implement dead letter queues for failed messages

05 - Data Transformation Layer [ENHANCED]

Implement ETL/ELT processing using Apache ecosystem tools:

A. Choose Your Transformation Approach:

- **ETL:** Transform before loading into Hadoop (use Apache Spark, Apache Flink, or Kafka Streams)
- **ELT:** Load raw data first, transform using Hive/Pig on Hadoop

B. Required Transformations:

- **Data Cleaning:**
 - Remove duplicate log entries
 - Handle missing fields with defaults
 - Validate timestamp formats and ranges
 - Filter noise and debug-level logs (configurable)
- **Data Enrichment:**
 - Parse and extract structured fields from unstructured logs
 - Perform GeoIP lookups for IP addresses in access logs
 - Add business context (map service IDs to service names)
 - Calculate derived metrics (error rates, latency percentiles)
- **Data Aggregation:**
 - Generate time-windowed aggregations (5-minute, hourly, daily rollups)
 - Calculate running statistics (moving averages, trend indicators)
 - Create summary tables for common query patterns

C. Apache Projects Integration:

- **Apache Spark:** For complex transformations and real-time processing
- **Apache Hive:** For SQL-based batch transformations and data warehousing
- **Apache Pig:** For dataflow scripting (alternative to Spark)
- **Apache Flume:** For additional log aggregation if needed
- **Apache HBase:** For indexed storage of transformed logs (optional)

D. Processing Requirements:

- Implement stateful processing where necessary (e.g., session tracking)
- Handle late-arriving data with watermarking strategies
- Ensure idempotent transformations for reprocessability
- Implement exactly-once semantics where critical

06 - Distributed Storage with Apache Hadoop [ENHANCED]

Configure Hadoop HDFS for scalable log storage:

- **Storage Architecture:**
 - Separate HDFS directories for:
 - /raw-logs/ - Original ingested logs
 - /processed-logs/ - Transformed and cleaned data
 - /aggregated-logs/ - Time-windowed summaries
 - /archive/ - Historical data beyond retention period
 - **Partitioning Strategy:**
 - Implement date-based partitioning: /logs/year=YYYY/month=MM/day=DD/
 - Add service-based sub-partitioning for large deployments
 - Use Hive-compatible partition naming conventions
 - **File Formats and Compression:**
 - Store raw logs in compressed text format (gzip)
 - Convert processed logs to columnar formats (Parquet or ORC) for query efficiency
 - Implement schema evolution support using Avro where appropriate
 - **HDFS Configuration:**
 - Configure replication factor (minimum 2, recommended 3)
 - Implement block size appropriate for log file sizes (128MB or 256MB)
 - Set up data locality for compute jobs
 - Configure short-circuit reads for performance
 - **Data Lifecycle Management:**
 - Implement retention policies (e.g., 90 days for detailed logs)
 - Archive older data to cheaper storage tiers
 - Implement automated cleanup scripts for expired data
-

07 - Reporting and Querying Capabilities [ENHANCED]

Develop comprehensive query and reporting mechanisms:

A. Query Interface Requirements:

- Implement Apache Hive for SQL-based querying
- Configure Apache Presto or Apache Drill for fast interactive queries (optional)
- Create pre-defined views for common query patterns

B. Time Window Queries:

- Last 24 hours: Real-time and recent log analysis
- Last 7 days: Weekly trend analysis and reporting
- Last 30 days: Monthly aggregations and long-term patterns
- Custom ranges: User-specified date/time windows

C. Query Capabilities:

- **Full-text search** across log messages
- **Field-based filtering** (service name, severity, user ID, etc.)
- **Aggregation queries** (count, sum, average, percentiles)
- **Time-series analysis** (trends, anomaly detection)

- **Join operations** across different log types
- **Export functionality** to CSV, JSON, or visualization tools

D. Performance Optimization:

- Create materialized views for frequent queries
- Implement column-level indexing
- Use predicate pushdown in Parquet/ORC files
- Configure query result caching
- Partition pruning for time-range queries

E. Visualization and Reporting (Optional Enhancement):

- Integrate Apache Superset or similar BI tool
 - Create dashboards showing:
 - Real-time log volume and error rates
 - Service health and uptime metrics
 - Top errors and exceptions by frequency
 - Request latency distributions
 - System resource utilization trends
-

Technical Deliverables

1. Infrastructure Code

- Docker Compose file OR Kubernetes manifests (YAML)
- Infrastructure-as-Code using Terraform or similar (optional)
- Service deployment scripts and automation

2. Configuration Files

- NiFi flow definitions (XML export)
- Kafka topic configurations
- Hadoop cluster configuration files (core-site.xml, hdfs-site.xml, etc.)
- Application configuration for microservices

3. Application Code

- Custom microservices source code (any language)
- Log generation scripts/libraries
- Data transformation jobs (Spark/Hive scripts)
- Query scripts and stored procedures

4. Documentation

- Architecture diagram showing all components and data flows
- Setup and deployment instructions

- API documentation for microservices
- Query examples and use cases
- Performance benchmarking results
- Troubleshooting guide

5. Demonstration Requirements

- Live demonstration of end-to-end pipeline operation
 - Show log generation from multiple services
 - Demonstrate data flowing through NiFi → Kafka → Hadoop
 - Execute sample queries across different time windows
 - Display monitoring dashboards (if implemented)
 - Demonstrate failure recovery (e.g., restart a service and show log continuity)
-

Evaluation Criteria

Technical Implementation (40%)

- Correct configuration of all Apache components
- Successful containerization and orchestration
- Proper implementation of data transformation logic
- Query performance and optimization

Architecture and Design (25%)

- Scalability considerations
- Fault tolerance and reliability measures
- Appropriate technology choices and justifications
- Clean separation of concerns

Data Quality and Pipeline Reliability (20%)

- Realistic and diverse log generation
- Data validation and error handling
- End-to-end data integrity
- Monitoring and observability

Documentation and Presentation (15%)

- Clear and comprehensive documentation
 - Architecture diagrams and explanations
 - Demonstration quality
 - Code organization and readability
-

Optional

1. **Security Implementation:** Add authentication and encryption (TLS) between components
 2. **Advanced Monitoring:** Implement distributed tracing with Apache SkyWalking or Zipkin
 3. **Machine Learning Integration:** Use Apache Spark MLlib for log anomaly detection
 4. **Real-time Alerting:** Implement Apache Flink for complex event processing and alerting
 5. **Multi-Region Deployment:** Deploy Kafka cluster across multiple "regions" (Docker networks/K8s namespaces)
 6. **Schema Registry:** Implement Confluent Schema Registry for schema management
 7. **Data Quality Checks:** Integrate Apache Griffin for data quality monitoring
 8. **Automated Scaling:** Implement horizontal pod autoscaling based on log volume (Kubernetes)
-

Recommended Timeline

- **Week 1:** Infrastructure setup (containers, networking, basic services)
 - **Week 2:** Microservices development and log generation implementation
 - **Week 3:** NiFi and Kafka integration, data ingestion
 - **Week 4:** Hadoop setup, data transformation, storage layer
 - **Week 5:** Query layer implementation, optimization, testing
 - **Week 6:** Documentation, final testing, demonstration preparation
-

Resources and References

- Apache NiFi Documentation: <https://nifi.apache.org/docs.html>
- Apache Kafka Documentation: <https://kafka.apache.org/documentation/>
- Apache Hadoop Documentation: <https://hadoop.apache.org/docs/>
- Docker Documentation: <https://docs.docker.com/>
- Kubernetes Documentation: <https://kubernetes.io/docs/>
- Apache Spark Documentation: <https://spark.apache.org/docs/latest/>

Note: Students are encouraged to work in teams for this comprehensive project. Good luck building your enterprise-grade data pipeline!